

Lecture 06 - Rasterization

This lecture explains how continuous primitives become discrete fragments on a grid. Rasterization is the bridge from geometric descriptions to sample-based image generation.

Source chunk	course_text_parts/03_lectures/06_rasterization.txt
Extracted slide pages	61
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Rasterization 1/2
Source cue	- Pixels ... / - ... are picture elements A Pixel Is NotA Little Square, / A Pixel Is NotA Little Square, / - ... are the smallest elements of a raster image / A Pixel Is NotA Little Square!

Professor-style explanation	The slide 'Rasterization 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Pixels ... / - ... are picture elements A Pixel Is NotA Little Square, / A Pixel Is NotA Little Square, / - ... are the smallest elements of a raster image / A Pixel Is NotA Little Square!. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Rasterization 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Pixels ... / - ... are picture elements A Pixel Is NotA Little Square, / A Pixel Is NotA Little Square, / - ... are the smallest elements of a raster image / A Pixel Is NotA Little Square!
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization 1/2'?

Page 3	Rasterization 2/2
Source cue	- Pixel-based displays require discretization of primitives into pixels / - Discretization is done by rasterization algorithms / - Rasterization is time-critical / - Avoid expensive operations / - Optimize for hardware implementation
Professor-style explanation	The slide 'Rasterization 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Pixel-based displays require discretization of primitives into pixels / - Discretization is done by rasterization algorithms / - Rasterization is time-critical / - Avoid expensive operations / - Optimize for hardware implementation. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Rasterization 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Pixel-based displays require discretization of primitives into pixels / - Discretization is done by rasterization algorithms / - Rasterization is time-critical / - Avoid expensive operations / - Optimize for hardware implementation

Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization 2/2'?
Page 4	6.1 Line Rasterization
Source cue	6.2 Triangle Edge Rasterization / 6.3 Region Filling / 6.4 Scanline-Based Triangle Rasterization / 6.5 Block-Based Triangle Rasterization
Professor-style explanation	The slide '6.1 Line Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: 6.2 Triangle Edge Rasterization / 6.3 Region Filling / 6.4 Scanline-Based Triangle Rasterization / 6.5 Block-Based Triangle Rasterization. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 6.1 Line Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: 6.2 Triangle Edge Rasterization / 6.3 Region Filling / 6.4 Scanline-Based Triangle Rasterization / 6.5 Block-Based Triangle Rasterization
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.1 Line Rasterization'?
Page 5	6.1 Line Rasterization
Source cue	Drawing a one pixel thick line
Professor-style explanation	The slide '6.1 Line Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Drawing a one pixel thick line. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about 6.1 Line Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Drawing a one pixel thick line
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.1 Line Rasterization'?

Page 6	Line Rasterization
Source cue	- Given two 2D endpoints in integer coordinates, / determine pixels depicting 1-pixel wide line segment between endpoints / - Trivial cases / - Horizontal line segments / - Vertical line segments
Professor-style explanation	The slide 'Line Rasterization' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Given two 2D endpoints in integer coordinates, / determine pixels depicting 1-pixel wide line segment between endpoints / - Trivial cases / - Horizontal line segments / - Vertical line segments. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Line Rasterization. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Given two 2D endpoints in integer coordinates, / determine pixels depicting 1-pixel wide line segment between endpoints / - Trivial cases / - Horizontal line segments / - Vertical line segments
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Line Rasterization'?

Page 7	Quality Criteria
Source cue	- For all line segments with a slope $ \Delta y / \Delta x \leq 1$, / one pixel should be drawn per column / - For all line segments with a slope $ \Delta y / \Delta x \geq 1$, / one pixel per line should be drawn / $ \Delta y / \Delta x > 1$

Professor-style explanation	The slide 'Quality Criteria' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - For all line segments with a slope $ \Delta y \leq 1$, / one pixel should be drawn per column / - For all line segments with a slope $ \Delta y \geq 1$, / one pixel per line should be drawn / $ \Delta y > 1$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Quality Criteria. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - For all line segments with a slope $ \Delta y \leq 1$, / one pixel should be drawn per column / - For all line segments with a slope $ \Delta y \geq 1$, / one pixel per line should be drawn / $ \Delta y > 1$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Quality Criteria'?

Page 8	Naïve Algorithm 1/2
Source cue	- Utilization of the Cartesian normal form of the straight line equation / $y = \Delta y \cdot x + \Delta y$ / - Assumptions (without restriction of generality) / - Line segments lie between (x, y) and (x, y) / $0 \leq x \leq 1$
Professor-style explanation	The slide 'Naïve Algorithm 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Utilization of the Cartesian normal form of the straight line equation / $y = \Delta y \cdot x + \Delta y$ / - Assumptions (without restriction of generality) / - Line segments lie between (x, y) and (x, y) / $0 \leq x \leq 1$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Naïve Algorithm 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Utilization of the Cartesian normal form of the straight line equation / $y = \Delta y \cdot x + \Delta y$ / - Assumptions (without restriction of generality) / - Line segments lie between (x, y) and (x, y) / $0 \leq x \leq 1$

Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Naïve Algorithm 1/2'?
Page 9	Naïve Algorithm 2/2
Source cue	- Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$)
Professor-style explanation	The slide 'Naïve Algorithm 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Naïve Algorithm 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Naïve Algorithm 2/2' into a causal sentence instead of repeating the slide title?
Page 10	Digital Difference Analyzer (DDA) 1/3
Source cue	- Reuse the last computed y to avoid multiplications in the loop / $x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + m)$ / void drawLine(int x0, int y0, int x1, int y1) { / float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope / float y = y0; // Start y at y0
Professor-style explanation	The slide 'Digital Difference Analyzer (DDA) 1/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Reuse the last computed y to avoid multiplications in the loop / $x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + m)$ / void drawLine(int x0, int y0, int x1, int y1) { / float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope / float y = y0; // Start y at y0. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.

Technical commentary	This slide is about Digital Difference Analyzer (DDA) 1/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Reuse the last computed y to avoid multiplications in the loop / $x \rightarrow x + 1 \Rightarrow (x, y) \rightarrow (x + 1, y + \blacksquare)$ / void drawLine(int x0, int y0, int x1, int y1) { / float m = float(y1 - y0) / float(x1 - x0); // Calculate the slope / float y = y0; // Start y at y0
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 1/3' into a causal sentence instead of repeating the slide title?

Page 11	Digital Difference Analyzer (DDA) 2/3
Source cue	- Algorithm can be extended to relax endpoint order / void drawLine(int x0, int y0, int x1, int y1) { / int sx = (x0 < x1 ? 1 : -1); // Determine x direction / float sm = sx * float(y1 - y0) / float(x1 - x0); // Calculate slope multiplied by step / float y = y0; // Start y at y0
Professor-style explanation	The slide 'Digital Difference Analyzer (DDA) 2/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Algorithm can be extended to relax endpoint order / void drawLine(int x0, int y0, int x1, int y1) { / int sx = (x0 < x1 ? 1 : -1); // Determine x direction / float sm = sx * float(y1 - y0) / float(x1 - x0); // Calculate slope multiplied by step / float y = y0; // Start y at y0. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Digital Difference Analyzer (DDA) 2/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Algorithm can be extended to relax endpoint order / void drawLine(int x0, int y0, int x1, int y1) { / int sx = (x0 < x1 ? 1 : -1); // Determine x direction / float sm = sx * float(y1 - y0) / float(x1 - x0); // Calculate slope multiplied by step / float y = y0; // Start y at y0
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 2/3' into a causal sentence instead of repeating the slide title?

Page 12	Digital Difference Analyzer (DDA) 3/3
Source cue	- Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ \blacksquare > 1$, since gaps arise / (solution: swap the roles of x and y for $ \blacksquare > 1$)

Professor-style explanation	The slide 'Digital Difference Analyzer (DDA) 3/3' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Digital Difference Analyzer (DDA) 3/3. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Downsides / - Does not work for $x = x / 0$ 1 / - Does not work for $ m > 1$, since gaps arise / (solution: swap the roles of x and y for $ m > 1$)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Digital Difference Analyzer (DDA) 3/3' into a causal sentence instead of repeating the slide title?

Page 13	Midpoint-Line Algorithm 1/2
Source cue	- Assumption 1: $0 \leq m \leq 1$ / (without loss of generality, as otherwise mirroring on axes/diagonals) / - Lower left end point $P_0 = (x_0, y_0)$ / 0 0 0 / upper right end point $P_1 = (x_1, y_1)$
Professor-style explanation	The slide 'Midpoint-Line Algorithm 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Assumption 1: $0 \leq m \leq 1$ / (without loss of generality, as otherwise mirroring on axes/diagonals) / - Lower left end point $P_0 = (x_0, y_0)$ / 0 0 0 / upper right end point $P_1 = (x_1, y_1)$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Midpoint-Line Algorithm 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Assumption 1: $0 \leq m \leq 1$ / (without loss of generality, as otherwise mirroring on axes/diagonals) / - Lower left end point $P_0 = (x_0, y_0)$ / 0 0 0 / upper right end point $P_1 = (x_1, y_1)$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Midpoint-Line Algorithm 1/2'?

Page 14	Midpoint-Line Algorithm 2/2
---------	-----------------------------

Source cue	- Observation / - $\blacksquare$ is closer to the line if $\blacksquare$ is above the line / - $\blacksquare\blacksquare$ is closer to the line if $\blacksquare$ is below the line / - Vertical distance between pixel to chose / and the actual line segment is always < 0.5
Professor-style explanation	The slide 'Midpoint-Line Algorithm 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Observation / - $\blacksquare$ is closer to the line if $\blacksquare$ is above the line / - $\blacksquare\blacksquare$ is closer to the line if $\blacksquare$ is below the line / - Vertical distance between pixel to chose / and the actual line segment is always < 0.5 . The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Midpoint-Line Algorithm 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Observation / - $\blacksquare$ is closer to the line if $\blacksquare$ is above the line / - $\blacksquare\blacksquare$ is closer to the line if $\blacksquare$ is below the line / - Vertical distance between pixel to chose / and the actual line segment is always < 0.5
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Midpoint-Line Algorithm 2/2'?
Page 15	Implicit Function
Source cue	- Line equation usually given as explicit function $\blacksquare(x) / \blacksquare x = y = \blacksquare \cdot x + \blacksquare = \cdot x + \blacksquare$, with $\blacksquare x = x - x$ and $\blacksquare y = y - y / 1 \blacksquare 1 \blacksquare$ / - Line equation can also be written as implicit function $\blacksquare(x, y) / \blacksquare x, y = \blacksquare \cdot x + \blacksquare \cdot y + \blacksquare = 0$
Professor-style explanation	The slide 'Implicit Function' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Line equation usually given as explicit function $\blacksquare(x) / \blacksquare x = y = \blacksquare \cdot x + \blacksquare = \cdot x + \blacksquare$, with $\blacksquare x = x - x$ and $\blacksquare y = y - y / 1 \blacksquare 1 \blacksquare$ / - Line equation can also be written as implicit function $\blacksquare(x, y) / \blacksquare x, y = \blacksquare \cdot x + \blacksquare \cdot y + \blacksquare = 0$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Implicit Function. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Line equation usually given as explicit function $f(x) / x = y = a \cdot x + b = c \cdot x + d$, with $a = x - x$ and $b = y - y / 1 - 1$ / - Line equation can also be written as implicit function $f(x, y) / x, y = a \cdot x + b \cdot y + c = 0$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Implicit Function'?

Page 16	Decision Variable D
Source cue	- Sign of D is sufficient to decide which point is chosen / - Define decision variable D as $D = D = D(x + 1, y + 1) / D$ / - If $D > 0$, select D (D below the line) / - If $D < 0$, select D (D above the line)
Professor-style explanation	The slide 'Decision Variable D ' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Sign of D is sufficient to decide which point is chosen / - Define decision variable D as $D = D = D(x + 1, y + 1) / D$ / - If $D > 0$, select D (D below the line) / - If $D < 0$, select D (D above the line). The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Decision Variable D . The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Sign of D is sufficient to decide which point is chosen / - Define decision variable D as $D = D = D(x + 1, y + 1) / D$ / - If $D > 0$, select D (D below the line) / - If $D < 0$, select D (D above the line)
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Decision Variable D ' into a causal sentence instead of repeating the slide title?

Page 17	Update of D for D
Source cue	- Previous decision variable was given by $D = D = D(x + 1, y + 1) = a \cdot x + b \cdot y + c / D = D = D(x + 1, y + 1) = a \cdot x + b \cdot y + c / 2$

Professor-style explanation	The slide 'Update of ■ for ■' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Previous decision variable was given by $\frac{1}{1} \cdot \frac{1}{\blacksquare} = \blacksquare \cdot x + 1, y + = \blacksquare \cdot x + 1 + \blacksquare \cdot y + + \blacksquare / \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare / 2 \ 2$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Update of ■ for ■. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Previous decision variable was given by $\frac{1}{1} \cdot \frac{1}{\blacksquare} = \blacksquare \cdot x + 1, y + = \blacksquare \cdot x + 1 + \blacksquare \cdot y + + \blacksquare / \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare / 2 \ 2$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Update of ■ for ■' into a causal sentence instead of repeating the slide title?

Page 18	Update of ■ for ■■
Source cue	- Previous decision variable was given by $\frac{1}{1} \cdot \frac{1}{\blacksquare} = \blacksquare \cdot x + 1, y + = \blacksquare \cdot x + 1 + \blacksquare \cdot y + + \blacksquare / \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare / 2 \ 2$
Professor-style explanation	The slide 'Update of ■ for ■■' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Previous decision variable was given by $\frac{1}{1} \cdot \frac{1}{\blacksquare} = \blacksquare \cdot x + 1, y + = \blacksquare \cdot x + 1 + \blacksquare \cdot y + + \blacksquare / \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare / 2 \ 2$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Update of ■ for ■■. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Previous decision variable was given by $\frac{1}{1} \cdot \frac{1}{\blacksquare} = \blacksquare \cdot x + 1, y + = \blacksquare \cdot x + 1 + \blacksquare \cdot y + + \blacksquare / \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare / 2 \ 2$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Update of ■ for ■■' into a causal sentence instead of repeating the slide title?

Page 19	Proceeding
Source cue	- In each step, the algorithm selects between two pixels ■■ and ■ / based on sign of decision variable ■ / - Depending on the choice, decision variable is updated / by adding difference $\Delta \blacksquare \blacksquare$ or $\Delta \blacksquare$

Professor-style explanation	The slide 'Proceeding' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - In each step, the algorithm selects between two pixels $\mathbf{p}$ and $\mathbf{q}$ / based on sign of decision variable $\mathbf{d}$ / - Depending on the choice, decision variable is updated / by adding difference $\Delta\mathbf{p}$ or $\Delta\mathbf{q}$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Proceeding. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - In each step, the algorithm selects between two pixels $\mathbf{p}$ and $\mathbf{q}$ / based on sign of decision variable $\mathbf{d}$ / - Depending on the choice, decision variable is updated / by adding difference $\Delta\mathbf{p}$ or $\Delta\mathbf{q}$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Proceeding' into a causal sentence instead of repeating the slide title?
Page 20	Initializing $\mathbf{p}$
Source cue	- First midpoint $\mathbf{p}$ is located at $(x + 1, y + 1) / 0011$ $\mathbf{p} = \mathbf{p} \cdot x + 1, y + = \mathbf{p} \cdot x + 1 + \mathbf{p} \cdot y + = \mathbf{p} \cdot x + \mathbf{p} \cdot y + \mathbf{p} + / 000000$
Professor-style explanation	The slide 'Initializing $\mathbf{p}$ ' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - First midpoint $\mathbf{p}$ is located at $(x + 1, y + 1) / 0011$ $\mathbf{p} = \mathbf{p} \cdot x + 1, y + = \mathbf{p} \cdot x + 1 + \mathbf{p} \cdot y + = \mathbf{p} \cdot x + \mathbf{p} \cdot y + \mathbf{p} + / 000000$. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Initializing $\mathbf{p}$. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - First midpoint $\mathbf{p}$ is located at $(x + 1, y + 1) / 0011$ $\mathbf{p} = \mathbf{p} \cdot x + 1, y + = \mathbf{p} \cdot x + 1 + \mathbf{p} \cdot y + = \mathbf{p} \cdot x + \mathbf{p} \cdot y + \mathbf{p} + / 000000$
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Initializing $\mathbf{p}$ ' into a causal sentence instead of repeating the slide title?
Page 21	Using $\mathbf{p}$
Source cue	- Midpoint line algorithms uses $\mathbf{p}$ as follows $/ \mathbf{p} = 2 \cdot \mathbf{p}y - \mathbf{p}x / \mathbf{p} \mathbf{p} \mathbf{p} \mathbf{p} \mathbf{p} / \Delta\mathbf{p} = 2 \cdot \mathbf{p}y - \mathbf{p}x / \Delta\mathbf{p} = 2 \cdot \mathbf{p}y$

Professor-style explanation	The slide 'Using ■' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Midpoint line algorithms uses ■ as follows $\frac{y}{x} = \frac{2 \cdot y_1 - y_0}{x_1 - x_0}$ / $\frac{y}{x} = \frac{2 \cdot y_1 - y_0}{x_1 - x_0}$ / $\frac{y}{x} = \frac{2 \cdot y_1 - y_0}{x_1 - x_0}$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Using ■. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Midpoint line algorithms uses ■ as follows $\frac{y}{x} = \frac{2 \cdot y_1 - y_0}{x_1 - x_0}$ / $\frac{y}{x} = \frac{2 \cdot y_1 - y_0}{x_1 - x_0}$ / $\frac{y}{x} = \frac{2 \cdot y_1 - y_0}{x_1 - x_0}$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Using ■'?
Page 22	Pseudo Code – Simple Version
Source cue	<pre>void midPointLine(int x0, int y0, int x1, int y1) { / int dx = x1 - x0; // Calculate delta x / int dy = y1 - y0; // Calculate delta y / int d = 2 * dy - dx; // Initial decision variable / int dE = 2 * dy; // Increment used for move to E</pre>
Professor-style explanation	The slide 'Pseudo Code – Simple Version' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: <pre>void midPointLine(int x0, int y0, int x1, int y1) { / int dx = x1 - x0; // Calculate delta x / int dy = y1 - y0; // Calculate delta y / int d = 2 * dy - dx; // Initial decision variable / int dE = 2 * dy; // Increment used for move to E.</pre> The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Pseudo Code – Simple Version. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: <pre>void midPointLine(int x0, int y0, int x1, int y1) { / int dx = x1 - x0; // Calculate delta x / int dy = y1 - y0; // Calculate delta y / int d = 2 * dy - dx; // Initial decision variable / int dE = 2 * dy; // Increment used for move to E</pre>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Pseudo Code – Simple Version' into a causal sentence instead of repeating the slide title?
Page 23	Example

Source cue	- Draw line from (5, 8) to (9, 11) / - Order of values for ■ / - 2, 0, 6, 4 / - Order of chosen pixels / - NE, E, NE, NE
Professor-style explanation	The slide 'Example' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Draw line from (5, 8) to (9, 11) / - Order of values for ■ / - 2, 0, 6, 4 / - Order of chosen pixels / - NE, E, NE, NE. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Example. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Draw line from (5, 8) to (9, 11) / - Order of values for ■ / - 2, 0, 6, 4 / - Order of chosen pixels / - NE, E, NE, NE
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Example'?
Page 24	Evaluation
Source cue	- Necessary arithmetic operations / - As only integer values are used, only integer operations are necessary / - Additions/subtractions only and multiplication by 2 / (left shift by 1 bit position) / - No multiplications in the loop
Professor-style explanation	The slide 'Evaluation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Necessary arithmetic operations / - As only integer values are used, only integer operations are necessary / - Additions/subtractions only and multiplication by 2 / (left shift by 1 bit position) / - No multiplications in the loop. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Evaluation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Necessary arithmetic operations / - As only integer values are used, only integer operations are necessary / - Additions/subtractions only and multiplication by 2 / (left shift by 1 bit position) / - No multiplications in the loop
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Evaluation' into a causal sentence instead of repeating the slide title?
Page 25	Pseudo Code - Reversible
Source cue	<pre>void drawLine(int x0, int y0, int x1, int y1) { / int dx = abs(x1 - x0); // Calculate absolute delta x / int sx = (x0 < x1 ? 1 : -1); // Set step x based on direction / int dy = abs(y1 - y0); // Calculate absolute delta y / int sy = (y0 < y1 ? 1 : -1); // Set step y based on direction</pre>
Professor-style explanation	The slide 'Pseudo Code - Reversible' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: <code>void drawLine(int x0, int y0, int x1, int y1) { / int dx = abs(x1 - x0); // Calculate absolute delta x / int sx = (x0 < x1 ? 1 : -1); // Set step x based on direction / int dy = abs(y1 - y0); // Calculate absolute delta y / int sy = (y0 < y1 ? 1 : -1); // Set step y based on direction</code>. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Pseudo Code - Reversible. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: <code>void drawLine(int x0, int y0, int x1, int y1) { / int dx = abs(x1 - x0); // Calculate absolute delta x / int sx = (x0 < x1 ? 1 : -1); // Set step x based on direction / int dy = abs(y1 - y0); // Calculate absolute delta y / int sy = (y0 < y1 ? 1 : -1); // Set step y based on direction</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Pseudo Code - Reversible' into a causal sentence instead of repeating the slide title?
Page 26	6.2 Triangle Edge Rasterization
Source cue	Drawing a one pixel thick triangle edge
Professor-style explanation	The slide '6.2 Triangle Edge Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Drawing a one pixel thick triangle edge. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 6.2 Triangle Edge Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Drawing a one pixel thick triangle edge

Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.2 Triangle Edge Rasterization'?
Page 27	Triangle Edges 1/2
Source cue	- Using midpoint line algorithm for triangle edge rasterization / - Provides pixels closest to triangle edge / - Does not consider if pixel lies inside or outside triangle / $\Rightarrow$ Edge pixels can penetrate into interior of adjacent triangles
Professor-style explanation	The slide 'Triangle Edges 1/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Using midpoint line algorithm for triangle edge rasterization / - Provides pixels closest to triangle edge / - Does not consider if pixel lies inside or outside triangle / $\Rightarrow$ Edge pixels can penetrate into interior of adjacent triangles. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Triangle Edges 1/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Using midpoint line algorithm for triangle edge rasterization / - Provides pixels closest to triangle edge / - Does not consider if pixel lies inside or outside triangle / $\Rightarrow$ Edge pixels can penetrate into interior of adjacent triangles
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Edges 1/2'?
Page 28	Triangle Edges 2/2
Source cue	- Midpoint line algorithm must consider inside vs. outside pixels / - Adaptation of the midpoint line algorithm / - Restriction to pixels strictly inside triangle / - Must be applied to line rasterization also for consistency reasons / - Definition of triangle inside:

Professor-style explanation	The slide 'Triangle Edges 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Midpoint line algorithm must consider inside vs. outside pixels - Adaptation of the midpoint line algorithm - Restriction to pixels strictly inside triangle - Must be applied to line rasterization also for consistency reasons - Definition of triangle inside. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Triangle Edges 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Midpoint line algorithm must consider inside vs. outside pixels - Adaptation of the midpoint line algorithm - Restriction to pixels strictly inside triangle - Must be applied to line rasterization also for consistency reasons - Definition of triangle inside:
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Edges 2/2'?

Page 29	Edge Rasterization 1/3
Source cue	- Triangle edge rasterization can exploit edge coherence - Two edges of triangle intersect a scanline - Same edges intersected by scanline  likely also intersect scanline  + 1 - Due to coherence, intersections can be calculated incrementally - For each increase of $y \rightarrow y + 1$
Professor-style explanation	The slide 'Edge Rasterization 1/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Triangle edge rasterization can exploit edge coherence - Two edges of triangle intersect a scanline - Same edges intersected by scanline  likely also intersect scanline  + 1 - Due to coherence, intersections can be calculated incrementally - For each increase of $y \rightarrow y + 1$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Edge Rasterization 1/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Triangle edge rasterization can exploit edge coherence / - Two edges of triangle intersect a scanline / - Same edges intersected by scanline ■ likely also intersect scanline ■ + 1 / - Due to coherence, intersections can be calculated incrementally / - For each increase of $y \rightarrow y + 1$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 1/3'?

Page 30	Edge Rasterization 2/3
Source cue	- Integer arithmetic can be used for (left) edges with slopes > 1 / - Consider $\Delta x \Delta y = (x_{\text{right}} - x_{\text{left}}) / (y_{\text{right}} - y_{\text{left}})$ / - x is only changed by $\Delta x \Delta y \Rightarrow x$ can be represented as a fraction with denominator $(y_{\text{right}} - y_{\text{left}})$ / - Let Δ be the integer part
Professor-style explanation	The slide 'Edge Rasterization 2/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Integer arithmetic can be used for (left) edges with slopes > 1 / - Consider $\Delta x \Delta y = (x_{\text{right}} - x_{\text{left}}) / (y_{\text{right}} - y_{\text{left}})$ / - x is only changed by $\Delta x \Delta y \Rightarrow x$ can be represented as a fraction with denominator $(y_{\text{right}} - y_{\text{left}})$ / - Let Δ be the integer part. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Edge Rasterization 2/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Integer arithmetic can be used for (left) edges with slopes > 1 / - Consider $\Delta x \Delta y = (x_{\text{right}} - x_{\text{left}}) / (y_{\text{right}} - y_{\text{left}})$ / - x is only changed by $\Delta x \Delta y \Rightarrow x$ can be represented as a fraction with denominator $(y_{\text{right}} - y_{\text{left}})$ / - Let Δ be the integer part
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 2/3'?

Page 31	Edge Rasterization 3/3
Source cue	<pre>void leftEdgeScan(int x0, int y0, int x1, int y1) { // slope > 1 / int d = y1 - y0; // denominator (Nenner) / int n = x1 - x0; // numerator (Zähler) / int a = x0; // integer part of x / int b = d; // fraction part of x plus d</pre>

Professor-style explanation	The slide 'Edge Rasterization 3/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: <code>void leftEdgeScan(int x0, int y0, int x1, int y1) { // slope > 1 / int d = y1 - y0; // denominator (Nenner) / int n = x1 - x0; // numerator (Zähler) / int a = x0; // integer part of x / int b = d; // fraction part of x plus d</code> . The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Edge Rasterization 3/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: <code>void leftEdgeScan(int x0, int y0, int x1, int y1) { // slope > 1 / int d = y1 - y0; // denominator (Nenner) / int n = x1 - x0; // numerator (Zähler) / int a = x0; // integer part of x / int b = d; // fraction part of x plus d</code>
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge Rasterization 3/3'?

Page 32	6.3 Region Filling
Source cue	Homogeneous coloring of regions
Professor-style explanation	The slide '6.3 Region Filling' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Homogeneous coloring of regions. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 6.3 Region Filling. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Homogeneous coloring of regions
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.3 Region Filling'?

Page 33	Region Types 1/2
---------	------------------

Source cue	- 4-connected / - All pixels of region can be reached by following movements / - Left, right, bottom, top / - 8-connected / - All pixels of region can be reached by following movements
Professor-style explanation	The slide 'Region Types 1/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - 4-connected / - All pixels of region can be reached by following movements / - Left, right, bottom, top / - 8-connected / - All pixels of region can be reached by following movements. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Region Types 1/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - 4-connected / - All pixels of region can be reached by following movements / - Left, right, bottom, top / - 8-connected / - All pixels of region can be reached by following movements
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Region Types 1/2' into a causal sentence instead of repeating the slide title?

Page 34	Region Types 2/2
Source cue	- Examples / 4-connected 8-connected 8-connected
Professor-style explanation	The slide 'Region Types 2/2' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Examples / 4-connected 8-connected 8-connected. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Region Types 2/2. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Examples / 4-connected 8-connected 8-connected
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Region Types 2/2' into a causal sentence instead of repeating the slide title?

Page 35	Region Spezifikation
Source cue	- Color-based / - Starting at pixel ■, determine largest contiguous region / which pixels have a specific color / - Color assignment done by flood fill algorithms / - Border-based

Professor-style explanation	The slide 'Region Spezifikation' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Color-based / - Starting at pixel ■, determine largest contiguous region / which pixels have a specific color / - Color assignment done by flood fill algorithms / - Border-based. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Region Spezifikation. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Color-based / - Starting at pixel ■, determine largest contiguous region / which pixels have a specific color / - Color assignment done by flood fill algorithms / - Border-based
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Region Spezifikation' into a causal sentence instead of repeating the slide title?

Page 36	Flood Fill Algorithm
Source cue	<pre>void floodFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / if (oldC == readPixel(x, y)) { // Check if the current color matches oldC / writePixel(x, y, newC); // Change the color of the pixel</pre>
Professor-style explanation	The slide 'Flood Fill Algorithm' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: void floodFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / if (oldC == readPixel(x, y)) { // Check if the current color matches oldC / writePixel(x, y, newC); // Change the color of the pixel. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Flood Fill Algorithm. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: void floodFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / if (oldC == readPixel(x, y)) { // Check if the current color matches oldC / writePixel(x, y, newC); // Change the color of the pixel
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

Check yourself	Can you turn 'Flood Fill Algorithm' into a causal sentence instead of repeating the slide title?
Page 37	Boundary Fill Algorithm
Source cue	<code>void boundaryFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / Color c = readPixel(x, y); / if (c == oldC) return; // Return if pixel is a boundary pixel</code>
Professor-style explanation	The slide 'Boundary Fill Algorithm' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: <code>void boundaryFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / Color c = readPixel(x, y); / if (c == oldC) return; // Return if pixel is a boundary pixel</code> . The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Boundary Fill Algorithm. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: <code>void boundaryFill4(int x, int y, Color oldC, Color newC) { / if ((x < 0) (x >= width)) return; // Return if x is out of bounds / if ((y < 0) (y >= height)) return; // Return if y is out of bounds / Color c = readPixel(x, y); / if (c == oldC) return; // Return if pixel is a boundary pixel</code>
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Boundary Fill Algorithm' into a causal sentence instead of repeating the slide title?
Page 38	Fill Algorithms
Source cue	- High number of recursions can lead to stack overflow / - Use smarter data structures, i.e., queues and stacks / Queue Stack
Professor-style explanation	The slide 'Fill Algorithms' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - High number of recursions can lead to stack overflow / - Use smarter data structures, i.e., queues and stacks / Queue Stack. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Fill Algorithms. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - High number of recursions can lead to stack overflow / - Use smarter data structures, i.e., queues and stacks / Queue Stack

Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Fill Algorithms' into a causal sentence instead of repeating the slide title?
Page 39	6.4 Scanline-Based Triangle Rasterization
Source cue	Segment-based rasterization of the simplest type of polygon
Professor-style explanation	The slide '6.4 Scanline-Based Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Segment-based rasterization of the simplest type of polygon. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 6.4 Scanline-Based Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Segment-based rasterization of the simplest type of polygon
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.4 Scanline-Based Triangle Rasterization'?
Page 40	Triangle Rasterization
Source cue	- Triangle rasterization / - Rasterize filled triangles is main task in real-time rendering / - Realization through graphics hardware desired / - General polygons split into triangles through tessellation / - Rasterization requires three steps
Professor-style explanation	The slide 'Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Triangle rasterization / - Rasterize filled triangles is main task in real-time rendering / - Realization through graphics hardware desired / - General polygons split into triangles through tessellation / - Rasterization requires three steps. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Triangle rasterization / - Rasterize filled triangles is main task in real-time rendering / - Realization through graphics hardware desired / - General polygons split into triangles through tessellation / - Rasterization requires three steps
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Triangle Rasterization'?

Page 41	Scanline-Based Rasterization
Source cue	- Rasterization through scanline conversion exploits spatial coherence / - Pixels of a scanline are colored similarly / - Adjacent scanlines span similar pixels / - Adjacent scanlines are colored similarly / - Scanline segments can be calculated
Professor-style explanation	The slide 'Scanline-Based Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Rasterization through scanline conversion exploits spatial coherence / - Pixels of a scanline are colored similarly / - Adjacent scanlines span similar pixels / - Adjacent scanlines are colored similarly / - Scanline segments can be calculated. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Scanline-Based Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Rasterization through scanline conversion exploits spatial coherence / - Pixels of a scanline are colored similarly / - Adjacent scanlines span similar pixels / - Adjacent scanlines are colored similarly / - Scanline segments can be calculated
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Scanline-Based Rasterization'?

Page 42	Algorithm Overview
Source cue	- Observation: triangle can be split into top and bottom half / - For each half two edge pixels and one segment exist / - Segments differ only slightly, depending on edge slope / - Procedure / - Sort triangle vertices based on y-coordinate

Professor-style explanation	The slide 'Algorithm Overview' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Observation: triangle can be split into top an bottom half / - For each half two edge pixels and one segment exist / - Segments differ only slightly, depending on edge slope / - Procedure / - Sort triangle vertices based on y-coordinate. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Algorithm Overview. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Observation: triangle can be split into top an bottom half / - For each half two edge pixels and one segment exist / - Segments differ only slightly, depending on edge slope / - Procedure / - Sort triangle vertices based on y-coordinate
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Algorithm Overview'?

Page 43	Slope Calculation
Source cue	- Calculate slope for each triangle edge: $1/\Delta = \Delta x/\Delta y$ / $\Delta = (x - x)/(y - y)$ / 02 2 0 2 0 / $\Delta = (x - x)/(y - y)$ / 01 1 0 1 0
Professor-style explanation	The slide 'Slope Calculation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Calculate slope for each triangle edge: $1/\Delta = \Delta x/\Delta y$ / $\Delta = (x - x)/(y - y)$ / 02 2 0 2 0 / $\Delta = (x - x)/(y - y)$ / 01 1 0 1 0. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Slope Calculation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Calculate slope for each triangle edge: $1/\Delta = \Delta x/\Delta y$ / $\Delta = (x - x)/(y - y)$ / 02 2 0 2 0 / $\Delta = (x - x)/(y - y)$ / 01 1 0 1 0
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.

Check yourself	Can you explain which samples/fragments are generated by 'Slope Calculation'?
Page 44	Pseudo Code
Source cue	<pre>void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) { / float XLeft = x0; // Start position for left x / float XRight = x0; // Start position for right x / int Y = y0; // Start position for y / float DeltaLeft = delta02; // Delta for left x for bottom half</pre>
Professor-style explanation	The slide 'Pseudo Code' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) { / float XLeft = x0; // Start position for left x / float XRight = x0; // Start position for right x / int Y = y0; // Start position for y / float DeltaLeft = delta02; // Delta for left x for bottom half. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Pseudo Code. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: void processTriangle(int x0, int y0, float delta02, float delta01, int Nbottom, int Ntop) { / float XLeft = x0; // Start position for left x / float XRight = x0; // Start position for right x / int Y = y0; // Start position for y / float DeltaLeft = delta02; // Delta for left x for bottom half
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Pseudo Code'?
Page 45	Segment Rasterization
Source cue	- Each segment generates fragments / - Rendering pipeline processes generated fragments / (x , y) / (e.g., depth testing, alpha blending) ■■■■■■ ■ / (x , y)

Professor-style explanation	The slide 'Segment Rasterization' explains rendering as a chain of object transformations. At the beginning there is scene information: models, vertices, primitives, attributes, camera state, and rendering state. The geometry part of the pipeline changes where the objects are and how they are represented; vertices become positioned data, vertices are assembled into primitives, and primitives are prepared for conversion to the image grid. On the slide, the concrete items are: - Each segment generates fragments / - Rendering pipeline processes generated fragments / (x , y) / (e.g., depth testing, alpha blending) ■■■■■■ ■ / (x , y). The important story is that the same scene information changes form several times. A triangle is first model data, then transformed geometry, then a projected primitive, then a set of fragments, and finally only some of those fragments become pixel updates. This is why the pipeline is not just a drawing diagram; it is the explanation for where image errors can enter.
Technical commentary	This slide is about Segment Rasterization. Scene or model data moves through a sequence of representations: vertices, primitives, fragments, tests, and framebuffer updates. Each named object is one stage in that conversion. Concrete items shown: - Each segment generates fragments / - Rendering pipeline processes generated fragments / (x , y) / (e.g., depth testing, alpha blending) ■■■■■■ ■ / (x , y)
Why it matters	Pipeline understanding lets you localize rendering errors instead of guessing randomly.
Check yourself	Can you name the input and output representation for 'Segment Rasterization' in the rendering pipeline?

Page 46	Vertex Attribute Interpolation
Source cue	- During rasterization vertex attributes (e.g., color, depth) / must be interpolated over triangle / (x , y , z) / ■■■■■■ ■ ■■■■■■ / - Bilinear interpolation can be used
Professor-style explanation	The slide 'Vertex Attribute Interpolation' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - During rasterization vertex attributes (e.g., color, depth) / must be interpolated over triangle / (x , y , z) / ■■■■■■ ■ ■■■■■■ / - Bilinear interpolation can be used. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Vertex Attribute Interpolation. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - During rasterization vertex attributes (e.g., color, depth) / must be interpolated over triangle / (x , y , z) / ■■■■■■ ■ ■■■■■■ / - Bilinear interpolation can be used

Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Vertex Attribute Interpolation'?
Page 47	Incremental Interpolation 1/3
Source cue	- Spatial coherence allows for an incremental attribute interpolation / - Example: consider triangle's plane equation for calculating z increments / - $a \cdot x + b \cdot y + c \cdot z + d = 0$ / - With plane coefficients a, b, c : $a, b, c = a - a \times (x - x_0) / 2 \cdot 0 \cdot 1 \cdot 2$
Professor-style explanation	The slide 'Incremental Interpolation 1/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Spatial coherence allows for an incremental attribute interpolation / - Example: consider triangle's plane equation for calculating z increments / - $a \cdot x + b \cdot y + c \cdot z + d = 0$ / - With plane coefficients a, b, c : $a, b, c = a - a \times (x - x_0) / 2 \cdot 0 \cdot 1 \cdot 2$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Incremental Interpolation 1/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Spatial coherence allows for an incremental attribute interpolation / - Example: consider triangle's plane equation for calculating z increments / - $a \cdot x + b \cdot y + c \cdot z + d = 0$ / - With plane coefficients a, b, c : $a, b, c = a - a \times (x - x_0) / 2 \cdot 0 \cdot 1 \cdot 2$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Incremental Interpolation 1/3'?
Page 48	Incremental Interpolation 2/3
Source cue	- Increments $\Delta z \Delta x$ and $\Delta z \Delta y$ are used to incrementally modify given z value, / i.e., when moving from (x, y) to $(x + 1, y)$ or $(x, y + 1)$ / - Case 1: next fragment in scanline (horizontal change by +1) / - $x \rightarrow x + 1 \Rightarrow z = z + \Delta z \Delta x / \Delta x \Delta y + 1$

Professor-style explanation	The slide 'Incremental Interpolation 2/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Increments Δz_x and Δz_y are used to incrementally modify given z value, / i.e., when moving from (x, y) to $(x + 1, y)$ or $(x, y + 1)$ / - Case 1: next fragment in scanline (horizontal change by +1) / - $x \rightarrow x + 1 \Rightarrow z = z + \Delta z_x / \Delta x + 1$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Incremental Interpolation 2/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Increments Δz_x and Δz_y are used to incrementally modify given z value, / i.e., when moving from (x, y) to $(x + 1, y)$ or $(x, y + 1)$ / - Case 1: next fragment in scanline (horizontal change by +1) / - $x \rightarrow x + 1 \Rightarrow z = z + \Delta z_x / \Delta x + 1$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Incremental Interpolation 2/3'?

Page 49	Incremental Interpolation 3/3
Source cue	- All vertex attributes can be interpolated incrementally analogous to depth / - Colors (r, g, b, a) / - Texture coordinates (u, v) / - ... / - For each attribute a define
Professor-style explanation	The slide 'Incremental Interpolation 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - All vertex attributes can be interpolated incrementally analogous to depth / - Colors (r, g, b, a) / - Texture coordinates (u, v) / - ... / - For each attribute a define. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Incremental Interpolation 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - All vertex attributes can be interpolated incrementally analogous to depth / - Colors (■, ■, ■, ■) / - Texture coordinates (■, ■) / - ... / - For each attribute ■ define
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Incremental Interpolation 3/3'?
Page 50	6.5 Tile-Based Triangle Rasterization
Source cue	Parallel tile-based triangle rasterization
Professor-style explanation	The slide '6.5 Tile-Based Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: Parallel tile-based triangle rasterization. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about 6.5 Tile-Based Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: Parallel tile-based triangle rasterization
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by '6.5 Tile-Based Triangle Rasterization'?
Page 51	Tile-Based Triangle Rasterization
Source cue	- Modern GPUs have a block-wise structure, / where one processor is responsible for one screen tile of adjacent pixels / - Segment-based triangled rasterization is difficult to parallelize / - Tile-based rasterization is a better fit with modern GPUs / - Advantage

Professor-style explanation	The slide 'Tile-Based Triangle Rasterization' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Modern GPUs have a block-wise structure, / where one processor is responsible for one screen tile of adjacent pixels / - Segment-based triangled rasterization is difficult to parallelize / - Tile-based rasterization is a better fit with modern GPUs / - Advantage. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Tile-Based Triangle Rasterization. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Modern GPUs have a block-wise structure, / where one processor is responsible for one screen tile of adjacent pixels / - Segment-based triangled rasterization is difficult to parallelize / - Tile-based rasterization is a better fit with modern GPUs / - Advantage
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Tile-Based Triangle Rasterization'?
Page 52	Edge-Based Triangle Representation 1/2
Source cue	- Edge-based area representation is possible through implicit edge functions / - Implicit edge function can be defined based on two edge vertices / $\blacksquare = (x, y)$ and $\blacksquare = (x, y) / 0 \ 0 \ 0 \ 1 \ 1 \ 1$ / - For a given coordinate (x, y), the implicit edge function is defined as
Professor-style explanation	The slide 'Edge-Based Triangle Representation 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. On the slide, the concrete items are: - Edge-based area representation is possible through implicit edge functions / - Implicit edge function can be defined based on two edge vertices / $\blacksquare = (x, y)$ and $\blacksquare = (x, y) / 0 \ 0 \ 0 \ 1 \ 1 \ 1$ / - For a given coordinate (x, y), the implicit edge function is defined as. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Edge-Based Triangle Representation 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. Concrete items shown: - Edge-based area representation is possible through implicit edge functions / - Implicit edge function can be defined based on two edge vertices / $\mathbf{e}_1 = (x_1, y_1)$ and $\mathbf{e}_2 = (x_2, y_2)$ / $0 \ 0 \ 0 \ 1 \ 1 \ 1$ / - For a given coordinate (x, y) , the implicit edge function is defined as
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Edge-Based Triangle Representation 1/2'?

Page 53	Edge-Based Triangle Representation 2/2
Source cue	- To represent a triangle, three implicit edge functions are combined / - $\mathbf{e}_1 \cdot \mathbf{x}, y = y_1 - y_1 \cdot x - x_1 \cdot y + \mathbf{e}_1 \cdot \mathbf{x} - x / 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0$ / - $\mathbf{e}_2 \cdot \mathbf{x}, y = y_2 - y_2 \cdot x - x_2 \cdot y + \mathbf{e}_2 \cdot \mathbf{x} - x / 1 \ 2 \ 2 \ 1 \ 2 \ 1 \ 1 \ 2 \ 2 \ 1$
Professor-style explanation	The slide 'Edge-Based Triangle Representation 2/2' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - To represent a triangle, three implicit edge functions are combined / - $\mathbf{e}_1 \cdot \mathbf{x}, y = y_1 - y_1 \cdot x - x_1 \cdot y + \mathbf{e}_1 \cdot \mathbf{x} - x / 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0$ / - $\mathbf{e}_2 \cdot \mathbf{x}, y = y_2 - y_2 \cdot x - x_2 \cdot y + \mathbf{e}_2 \cdot \mathbf{x} - x / 1 \ 2 \ 2 \ 1 \ 2 \ 1 \ 1 \ 2 \ 2 \ 1$. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Edge-Based Triangle Representation 2/2. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - To represent a triangle, three implicit edge functions are combined / - $\mathbf{e}_1 \cdot \mathbf{x}, y = y_1 - y_1 \cdot x - x_1 \cdot y + \mathbf{e}_1 \cdot \mathbf{x} - x / 0 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0$ / - $\mathbf{e}_2 \cdot \mathbf{x}, y = y_2 - y_2 \cdot x - x_2 \cdot y + \mathbf{e}_2 \cdot \mathbf{x} - x / 1 \ 2 \ 2 \ 1 \ 2 \ 1 \ 1 \ 2 \ 2 \ 1$
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Edge-Based Triangle Representation 2/2'?

Page 54	Rasterization Procedure
Source cue	1. Determine tiles which are (partially) covering triangle / (e.g., easiest approximation would be bounding box) / 2. For each covering tile / - For each pixel / - If all implicit edge functions evaluate to < 0 2

Professor-style explanation	The slide 'Rasterization Procedure' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: 1. Determine tiles which are (partially) covering triangle / (e.g., easiest approximation would be bounding box) / 2. For each covering tile / - For each pixel / - If all implicit edge functions evaluate to < 0 2. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Rasterization Procedure. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: 1. Determine tiles which are (partially) covering triangle / (e.g., easiest approximation would be bounding box) / 2. For each covering tile / - For each pixel / - If all implicit edge functions evaluate to < 0 2
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Rasterization Procedure'?
Page 55	between adjacent triangles
Source cue	- Pixel-based displays require the rasterization of primitives / - Rasterization fidelity is critical for image quality / - Rasterization is time-critical and should be optimized for hardware / - Focus on triangles due to speed and ease of tessellation of other polygons / - Segment-based triangle rasterization avoids overlap
Professor-style explanation	The slide 'between adjacent triangles' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. On the slide, the concrete items are: - Pixel-based displays require the rasterization of primitives / - Rasterization fidelity is critical for image quality / - Rasterization is time-critical and should be optimized for hardware / - Focus on triangles due to speed and ease of tessellation of other polygons / - Segment-based triangle rasterization avoids overlap. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.

Technical commentary	This slide is about between adjacent triangles. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. Concrete items shown: - Pixel-based displays require the rasterization of primitives / - Rasterization fidelity is critical for image quality / - Rasterization is time-critical and should be optimized for hardware / - Focus on triangles due to speed and ease of tessellation of other polygons / - Segment-based triangle rasterization avoids overlap
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'between adjacent triangles'?

Page 56	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. On the slide, the concrete items are: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Literature and other sources used in this chapter' into a causal sentence instead of repeating the slide title?

Page 57	Practice, Addison-Wesley.
Source cue	- Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Research Publications / - Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter.

Professor-style explanation	The slide 'Practice, Addison-Wesley.' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Research Publications / - Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Practice, Addison-Wesley.. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: - Text Books / - Foley J., van Dam, A., Feiner, S. (2013). Computer Graphics: Principles and / Practice, Addison-Wesley. / - Research Publications / - Bresenham, J. E. (1998). Algorithm for computer control of a digital plotter.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Practice, Addison-Wesley.' into a causal sentence instead of repeating the slide title?

Page 58	Slide for Future: Sustainability in CG (@ EG25)
Source cue	also @ IEEE VR
Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: also @ IEEE VR. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: also @ IEEE VR
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?

Page 59	Slide for Future: Sustainability in CG (@ EG25)
Source cue	Unoptimized Optimized Unoptimized Optimized / Inverse Simulation of Radiative Thermal Transport / Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn

Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Unoptimized Optimized Unoptimized Optimized / Inverse Simulation of Radiative Thermal Transport / Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Unoptimized Optimized Unoptimized Optimized / Inverse Simulation of Radiative Thermal Transport / Christian Freude, Lukas Lipp, Matthias Zezulka, Florian Rist, Michael Wimmer, David Hahn
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?
Page 60	Slide for Future: Sustainability in CG (@ EG25)
Source cue	Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints / Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi
Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints / Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Optimizing Free-Form Grid Shells with Reclaimed Elements under Inventory Constraints / Andrea Favilli, Francesco Laccone, Paolo Cignoni, Luigi Malomo, Daniela Giorgi
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?
Page 61	Slide for Future: Sustainability in CG (@ EG25)

Source cue	Stress-Aligned Hexahedral Lattice Structures / Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann
Professor-style explanation	The slide 'Slide for Future: Sustainability in CG (@ EG25)' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. On the slide, the concrete items are: Stress-Aligned Hexahedral Lattice Structures / Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about Slide for Future: Sustainability in CG (@ EG25). The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. Concrete items shown: Stress-Aligned Hexahedral Lattice Structures / Dennis Bukenberger, Junpeng Wang, Jun Wu, Rüdiger Westermann
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Slide for Future: Sustainability in CG (@ EG25)' into a causal sentence instead of repeating the slide title?

06.1 Line Rasterization

Source location: Section 6.1

Commentary

A mathematical line has infinitely many points, but a raster display has a finite grid. Line rasterization decides which grid cells or samples best approximate the ideal line. The algorithm must balance accuracy, speed, and consistency.

Incremental algorithms avoid recomputing expensive formulas for every pixel. The important idea is to step along one axis and update an error term that decides when to step along the other axis.

Mental model

Rasterizing a line means approximating a continuous path by grid decisions.

Check yourself

Why are incremental error updates useful in line rasterization?

06.2 Triangle Edge Rasterization

Source location: Section 6.2

Commentary

Triangles are the core primitive in real-time rendering. Edge functions or half-space tests decide whether a sample lies inside the triangle. This allows the rasterizer to test coverage systematically.

Coverage is not final visibility. A covered sample becomes a fragment candidate. Depth testing, stencil testing, blending, and other operations still decide whether the framebuffer changes.

Mental model

Triangle rasterization answers the question: which samples are inside this projected triangle?

Check yourself

What later stage can reject a fragment after triangle coverage succeeds?

06.3 Region Filling

Source location: Section 6.3

Commentary

Region filling generalizes the idea of determining interior samples. The algorithm must identify connected areas or spans that should receive color. This connects rasterization to scan conversion and image-space reasoning.

The practical issue is avoiding gaps, double fills, or inconsistent boundary handling. Small choices at edges can become visible artifacts when many primitives meet.

Mental model

Filling is about turning boundary descriptions into consistent interior samples.

Check yourself

Why can inconsistent edge rules create cracks between adjacent primitives?

06.4 Scanline-Based Triangle Rasterization

Source location: Section 6.4

Commentary

Scanline rasterization processes horizontal rows of the image. For each row that crosses a triangle, the algorithm finds the covered interval and fills the span. Attribute interpolation can be updated along edges and across each span.

The method is intuitive because it follows the memory layout of many images. It also demonstrates why interpolation is tied to rasterization: once you know a sample location inside the primitive, you can interpolate depth, color, normals, or texture coordinates.

Mental model

Each scanline asks: where does this triangle enter and leave this row?

Check yourself

Which attributes might be interpolated while filling triangle spans?

06.5 Tile-Based Triangle Rasterization

Commentary

Tile- or block-based approaches group pixels into small regions. This can improve locality and parallel work distribution. Modern GPUs often reason in blocks or tiles internally because rendering is massively parallel.

For the exam, connect this to efficiency. The mathematical goal is still coverage, but the implementation is organized to use hardware resources well.

Mental model

Block-based rasterization keeps the same coverage problem but changes the work organization.

Check yourself

Why is block organization attractive for parallel graphics hardware?

End-of-lecture summary

If you remember only one thing from Lecture 06, remember this: This lecture explains how continuous primitives become discrete fragments on a grid. Rasterization is the bridge from geometric descriptions to sample-based image generation.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.